

Des données, et des tests

PostgreSQL en conteneur, une authentification volontairement simplifiée.

Toujours la même règle : branche, MR, merge. Aucune exception.

Six étapes, dont deux sur les tests.

LES DONNÉES

PostgreSQL dans un conteneur, deux tables, une watchlist qui survit au redémarrage.

L'AUTH

Une watchlist par utilisateur.

LES TESTS

Ce sont eux qui autoriseront le déploiement en séance 3.

01

01 / 06 : 15 MIN

PostgreSQL dans un conteneur

FICHER `docker-compose.yml`

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: binggge
      POSTGRES_PASSWORD: binggge
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

TERMINAL

```
$ docker compose up -d
$ docker compose ps
$ docker compose logs -f db
$ cd api && npm i pg
```

Le volume `pgdata` est la seule ligne qui empêche vos données de disparaître. Testez : down puis up.

02

02 / 06 : 12 MIN

Deux tables, versionnées

FICHIER `api/db/schema.sql`

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  login TEXT UNIQUE NOT NULL  
);  
  
CREATE TABLE watchlist (  
  id SERIAL PRIMARY KEY,  
  user_id INT REFERENCES users(id),  
  show_id INT NOT NULL,  
  title TEXT NOT NULL,  
  seen BOOLEAN DEFAULT false  
);
```

TERMINAL : appliquer le schéma

```
$ docker compose exec -T db \  
  psql -U postgres binggge \  
  < api/db/schema.sql  
  
puis vérifier  
  
$ docker compose exec db \  
  psql -U postgres binggge -c "\dt"
```

Le schéma est un fichier du dépôt, pas une manipulation dans un outil graphique. Il se relit et se déploie.

03

03 / 06 : 10 MIN

Un utilisateur, et rien de plus

```
FICHIER  api/src/server.js
// POST /register
await db.query(
  'INSERT INTO users(login) VALUES($1)',
  [req.body.login])
// le garde des routes privées
const user = (req, res, next) =>
  req.get('X-User')
    ? next()
    : res.status(401).json({ error: 'non
authentifié' })
app.get('/watchlist', user, ...)
```

À QUOI SERT CET EN-TÊTE

À ce que /watchlist puisse dire non. Sans route qui refuse, vous n'avez aucun cas de refus à tester.

Ni mot de passe, ni hash, ni jeton. L'en-tête X-User nomme l'utilisateur, c'est tout.

Une ligne dans le README : « pas d'authentification réelle, l'en-tête X-User tient lieu d'identité ». C'est cette ligne qui est notée, pas le code.

04

04 / 06 : 15 MIN

La watchlist devient privée

TERMINAL : les trois appels à vérifier avant de passer aux tests

```
$ curl -X POST localhost:3000/register -H 'content-type: application/json' \
  -d '{"login":"olivia"}'
$ curl -X POST localhost:3000/watchlist -H 'X-User: olivia' \
  -H 'content-type: application/json' -d '{"show_id":44778,"title":"Severance"}'
$ curl localhost:3000/watchlist # sans en-tête : doit renvoyer 401
```

Un test, c'est une autorisation de déployer

Sans tests, chaque déploiement demande votre attention et celle de quelqu'un d'autre.
Avec des tests, la machine décide.

Aujourd'hui

Vous les écrivez et les lancez à la main.

Séance 3

Le pipeline les lance. Rouge, la MR est bloquée.

Séance 4

Après l'incident, vous en écrivez un qui reproduit le bug.

05

05 / 06 : 35 MIN

Quatre tests qui passent

TERMINAL

```
$ cd api && npm i -D supertest  
$ npm test
```

FICHIER api/package.json

```
"test": "node --test tests/"
```

FICHIER api/tests/server.test.js

```
test('health répond ok', async () => {  
  const r = await request(app).get('/health')  
  assert.equal(r.status, 200)  
})
```

01 /health répond 200.

02 Une inscription crée bien l'utilisateur.

03 Ajouter une série la fait apparaître dans /watchlist.

04 /watchlist sans en-tête renvoie 401.

Le quatrième vérifie un refus. C'est celui auquel personne ne pense, et c'est souvent celui qui sauve.

06

06 / 06 : 10 MIN

Tenez votre tableau

Vos huit tickets de la séance 1 existent toujours. Les quatre du jalon « Séance 2 » doivent finir en « fait » ce soir.

DANS GITLAB : description de la MR

Ajoute l'authentification et la watchlist
par utilisateur.

Closes #4, #5

les deux tickets se ferment au merge

Montrez-moi un test rouge

Votre API accepte d'ajouter une série avec un titre vide. Écrivez le test qui le prouve, montrez-le rouge, corrigez, montrez-le vert.

Prenez des captures d'écrans et intégrez les à votre dépôt avec nom :

TEST_WATCHLIST_NOK.jpeg => test rouge

TEST_WATCHLIST_OK.jpeg => test vert

```
curl -X POST localhost:3000/watchlist \  
  -H 'X-User: olivia' -d \  
  '{"show_id":1,"title":""}'
```

Rouge d'abord, vert ensuite. Un test écrit après le correctif ne prouve rien.

CE QUI SERA REGARDÉ

Je note votre dépôt, pas votre démo.

GIT : 5 PTS

Commits réguliers, messages lisibles. Une branche et une MR mergée par étape. .env hors dépôt.

API : 5 PTS

La base démarre avec compose up. Une route lit, une route écrit. /watchlist refuse sans X-User.

TESTS : 5 PTS

Tests verts sur les quatre demandés, dont le refus. Défi : le commit du test rouge précède celui du correctif.

PROJET : 5 PTS

Tableau tenu, une seule carte en cours. Chaque MR liée à son ticket. README à jour sur l'auth.

Pas fini ? Chaque étape validée compte. Un dépôt à moitié fait vaut mieux qu'un dépôt vide.

Avant de fermer la machine

01

`docker compose up -d` puis `npm test` : quatre tests verts.

02

Ma watchlist survit à un `compose down` suivi d'un `up`.

03

Le README dit qu'il n'y a pas d'authentification réelle, et pourquoi.

04

Deux MR de plus, mergées, chacune liée à son ticket par `Closes #n`.

Séance 3 : Docker, le pipeline, et votre première mise en ligne.